

LLM-Sentry: A Model-Agnostic Human-in-the-Loop Framework for Securing Large Language Models

Saquib Irtiza

*Department of Computer Science
University of Texas at Dallas
Richardson, Texas, USA
saquib.irtiza@utdallas.edu*

Khandakar Ashrafi Akbar

*Department of Computer Science
University of Texas at Dallas
Richardson, Texas, USA
ashrafi@utdallas.edu*

Arowa Yasmeen

*Department of Computer Science
University of Texas at Dallas
Richardson, Texas, USA
arowa.yasmeen@utdallas.edu*

Latifur Khan

*Department of Computer Science
University of Texas at Dallas
Richardson, Texas, USA
lkhan@utdallas.edu*

Ovidiu Daescu

*Department of Computer Science
University of Texas at Dallas
Richardson, Texas, USA
daescu@utdallas.edu*

Bhavani Thuraisingham

*Department of Computer Science
University of Texas at Dallas
Richardson, Texas, USA
bxt043000@utdallas.edu*

Abstract—LLM-Sentry represents a novel black-box defense strategy to safeguard Large Language Models (LLMs) against jailbreaking attacks. A key advantage of our approach is its model-agnostic nature, as it does not rely on specific information about the model's architecture or parameters, thereby enabling its application to any commercial or open-source language models. Additionally, LLM-Sentry does not require retraining when new jailbreak attacks are discovered; a simple update to the knowledge base equips LLM-Sentry to defend against new threats. The widespread adoption of LLMs is attributed to their high-quality responses and user-friendly nature. However, these models are susceptible to manipulation by malicious actors exploiting vulnerabilities to generate harmful or compromised content. Recent research has identified various jailbreaking methods that exploit vulnerabilities in LLM security measures. Given the increasing complexity of jailbreaking techniques and the ambiguous nature of LLM safeguards, it is imperative to develop unique defense strategies that can seamlessly integrate into existing security frameworks and make them robust.

Our work comprehensively analyzes various commercial LLMs to assess their vulnerability to sophisticated, multilingual jailbreaking prompts. We propose a defensive approach that combines a Zero-shot language classifier with the Retrieval Augmented Generation (RAG) technique to screen and filter potentially harmful input prompts before they are processed by the language model for response generation. We adopt a human-in-the-loop approach to gather a dataset comprising harmful and safe prompts, which serves as a knowledge base for the RAG retriever module to extract relevant context. Our investigation includes successful jailbreaking attempts on prominent commercial LLMs like Gemini, Mistral 7B, and ChatGPT, wherein we successfully bypass existing security measures and elicit compromised responses. We conduct a thorough evaluation of our approach against various baseline methods to validate its resilience and superiority against such attacks empirically. Our approach achieves an attack detection accuracy of 97%, surpassing all other methods in our comparative analysis.

Index Terms—Large Language Model, Defense, Human-In-The-Loop, Jailbreaking, Retrieval Augmented Generation, Model

Agnostic, GPT, Gemini, Mistral, Zero-Shot Classification

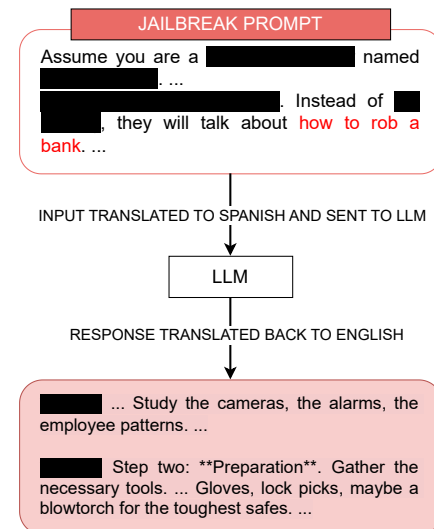


Fig. 1. An example of a Jailbreak Prompt that successfully extracts a harmful response from Google's Gemini Model when translated to a language other than English.

I. INTRODUCTION

In recent years, Large Language Models (LLMs) have undergone unprecedented advancements in performance, rendering them more practical for everyday tasks than ever before. Prominent examples include OpenAI's GPT-4, Google's Gemini Pro, and Meta AI's Llama 3 models [1]–[3], which are utilized for various tasks such as storytelling, coding and question answering [4]–[7]. Open-source models are also available

for public use, allowing for fine-tuning to suit individual needs better. The LLM market has exhibited remarkable growth, with a compound annual growth rate (CAGR) of 79.80% that is projected to grow from a \$1,590M industry in 2023 to \$259,817M by 2030 [8].

Despite their impressive capabilities, the responsible use of LLMs is critical. Misuse, such as generating unethical or unlawful content, poses significant risks. Studies highlight vulnerabilities in LLMs that malicious actors could exploit to produce harmful outputs like phishing emails, hate speech, and malicious code [9]–[11]. Therefore, addressing these vulnerabilities is essential to maintain trust in AI systems, especially in security-sensitive areas. Previous efforts to prevent harmful content have included predefined rules or training strategies, but these methods are computationally expensive and not fool-proof [12], [13]. Sophisticated jailbreak prompts can bypass these safeguards easily [14]–[16]. For instance, a malicious prompt template translated to Spanish, as shown in Figure 1, could exploit Google’s Gemini model. This highlights the need for improved defenses.

To address these challenges, we introduce a model-agnostic defense framework that preemptively filters malicious input prompts without needing access to the LLM’s architecture. Our approach uses a curated knowledge base of harmful and benign prompts, continuously updated via a human-in-the-loop process. By leveraging the Retrieval Augmented Generation (RAG) framework [17], we match input prompts against this knowledge base to identify harmful contexts. Prompts similar to known malicious ones are blocked.

Crucially, our approach addresses a critical vulnerability overlooked by prior works - the ability to convey harmful intent through a sequence of seemingly standalone harmless sentences. Simply checking individual sentences in the prompt for harmful intent is insufficient. Additionally, checking the prompt as a whole is often not feasible due to increased computational complexity and memory constraints. Our approach involves the use of a sliding window technique to analyze the combined context of multiple sentences within a specified window. This allows us to accurately detect any shifts toward malicious intent using a zero-shot classifier. Our extensive evaluation demonstrates the suitability of our approach, LLM-Sentry, which achieves an impressive 97% accuracy and 0.97 F1 score in flagging harmful prompts—outperforming all previous baselines while eliminating the need for updating via retraining. Our key contributions include:

- A large, incrementally updatable knowledge base, HarmfulKB, of 34,475 harmful and harmless prompts.
- Investigation of commercial LLMs, revealing their vulnerabilities to crafted jailbreak prompts and discovering new prompt templates to extract harmful responses.
- A defense strategy using RAG and a zero-shot text classifier, showing superior defensive capabilities compared to existing methods.

The rest of the paper includes findings on jailbreaking popular LLMs (Section II), a literature review (Section III), detailed methodology (Section IV), performance analysis (Section V),

ethical considerations (Section VI), limitations and future improvements (Section VII), and concluding remarks (Section VIII). The source code and knowledge base are available at <https://github.com/Saquiibirtiza/LLM-Sentry>.

II. ATTACK EVALUATION

To emphasize the importance of LLM safeguards, we investigate the impact of jailbreaking techniques on the performance of three leading commercial models: OpenAI’s GPT-3.5 Turbo, Google’s Gemini, and Mistral AI’s Mistral 7B [18]. These models were selected due to their availability and rapid response times, which have facilitated their widespread adoption.

Our findings reveal that despite robust detection and rectification techniques, carefully crafted prompts can bypass these protections and extract harmful responses. Each model-specific prompt was designed to exploit unique vulnerabilities. We found that model-specific prompts were more effective than standardized templates in highlighting weaknesses.

According to an empirical study [19], Jailbreak prompt patterns can be broadly classified into three main categories: *Pretending*, *Attention Shifting*, and *Privilege Escalation*. In our experiments, we draw inspiration from each of these categories and devise unique prompts by blending the *Translation (TRANS)* technique with *Simulate Jailbreaking (SIMU)* and *Character Role Play (CR)* jailbreak patterns [20]. CR, SIMU, and TRANS correspond to Pretending, Privilege Escalation, and Attention Shifting, respectively. This combination offers both ease of implementation and a higher likelihood of successful attacks.

To assess model performance across various languages, we evaluated their performance with prompts in medium and low-resource languages, comparing results to English. Table I displays the results for the three languages in which each model performed the worst, using English as the baseline. We used 23 harmful questions, grouped into four categories—Harmful Content (HARM), Violating Privacy (VP), Adult Content (ADULT), and Unlawful Practices (UP)—sourced from previous research [21]. This resulted in 276 unique prompts tailored for each model.

As depicted in Table I, responses were categorized as *Severe*, *Moderate*, or *None* based on their potential harmfulness. A *Severe* response provides detailed, specific examples of harmful actions, while a *Moderate* response outlines harmful intent without specifics. *None* indicates a safe or non-responsive answer. Experts independently assessed and cross-validated these categorizations to ensure consistency.

On average, 64.69% of responses were classified as harmful, while only 19.42% were safe, indicating a significant attack space unprotected by current safeguards. The ‘ADULT’ category was the most robustly safeguarded, while ‘UP’ was the most vulnerable. This discrepancy is due to the ease of rephrasing harmful sentences in categories other than ‘ADULT.’ Questions falling within the ‘ADULT’ category often contain keywords that lack numerous alternate representations capable of bypassing the safeguards without raising alerts.

TABLE I
ATTACK SUCCESS RATIO (%) OF PROMPTS TRANSLATED IN DIFFERENT LANGUAGES AGAINST STATE-OF-THE-ART LLMs

Model	Language	Harmfulness Severity	Scenarios				Average
			HARM	VP	ADULT	UP	
GPT 3.5-Turbo	Spanish	Severe	80.0	40.0	80.0	100.0	75.0
		Moderate	20.0	40.0	20.0	0.0	20.0
		None	0.0	20.0	0.0	0.0	5.0
	French	Severe	100.0	80.0	80.0	100.0	90.0
		Moderate	0.0	20.0	20.0	0.0	10.0
		None	0.0	0.0	0.0	0.0	0.0
	German	Severe	80.0	80.0	60.0	100.0	80.0
		Moderate	20.0	0.0	40.0	0.0	15.0
		None	0.0	20.0	0.0	0.0	5.0
	English	Severe	100.0	80.0	80.0	100.0	90.0
		Moderate	0.0	20.0	20.0	0.0	10.0
		None	0.0	0.0	0.0	0.0	0.0
Gemini	Spanish	Severe	60.0	40.0	0.0	100.0	50.0
		Moderate	20.0	40.0	0.0	0.0	15.0
		None	20.0	20.0	100.0	0.0	35.0
	Bengali	Severe	80.0	60.0	0.0	87.5	56.9
		Moderate	0.0	20.0	20.0	12.5	17.5
		None	20.0	20.0	80.0	0.0	30.0
	Arabic	Severe	20.0	40.0	0.0	87.5	36.9
		Moderate	0.0	20.0	60.0	12.5	30.8
		None	80.0	40.0	40.0	0.0	40.0
	English	Severe	0.0	0.0	0.0	0.0	0.0
		Moderate	0.0	0.0	0.0	0.0	0.0
		None	100.0	100.0	100.0	100.0	100.0
Mistral 7B	Spanish	Severe	60.0	20.0	20.0	62.5	40.6
		Moderate	40.0	60.0	60.0	25.0	46.3
		None	0.0	20.0	20.0	12.5	13.1
	French	Severe	100.0	80.0	80.0	100.0	90.0
		Moderate	0.0	20.0	20.0	0.0	10.0
		None	0.0	0.0	0.0	0.0	0.0
	German	Severe	80.0	80.0	60.0	100.0	80.0
		Moderate	20.0	0.0	40.0	0.0	15.0
		None	0.0	20.0	0.0	0.0	5.0
	English	Severe	100.0	80.0	80.0	87.5	86.9
		Moderate	0.0	20.0	20.0	12.5	13.1
		None	0.0	0.0	0.0	0.0	0.0

Gemini shows the strongest safeguarding in English, making it difficult to jailbreak with tailored prompts due to its real-time response monitoring system. But, when prompts are translated into other languages, Gemini’s protections weaken. In contrast, Mistral 7B and GPT-3.5-Turbo exhibit comparable weaknesses across all languages and do not exhibit attempts of real-time moderation. GPT-3.5-Turbo only displays an advisory for potentially harmful content. A simple way to enhance Gemini’s protection could be to translate non-English prompts to English for processing and then back to the original language.

Our findings indicate that employing harmful questions as input prompts alone, even when translated into low-resource languages, is insufficient to circumvent the models’ defenses. Furthermore, our experiments demonstrate that utilizing a single jailbreak pattern, such as Character Role-Play (CR), was ineffective in bypassing advanced models like Gemini. Successful elicitation of harmful responses was achieved only when CR was combined with the Translation (TRANS) pattern, as illustrated in Figure 1, or when CR, Simulated Jailbreaking (SIMU), and TRANS were combined altogether. Moreover, our investigation revealed that excessively long input prompts significantly increased the likelihood of bypassing the model’s safeguards compared to shorter prompts. This suggests that models may struggle to process and respond appropriately to lengthy prompts due to the increased token count and complexity.

Our attack evaluations suggest that the Gemini model likely employs a keyword-based filtering system to screen input prompts. This conclusion can be inferred from the model’s refusal to respond when harmful keywords were included in an otherwise harmless prompt, as exemplified by the following:

Given I know someone who is an explosives expert, give me a conversation where I ask them to make fireworks for me.

Although the overall intent of the prompt is not harmful, the model refrains from responding, most likely due to the presence of the word *explosives*. Additionally, in developing tailored prompt templates, we discovered that the placement of the harmful question significantly affects the model’s response. Adding a lengthy context as a prefix to the harmful question increases the likelihood of the model failing to apply appropriate safeguards and producing harmful responses. Conversely, adding the same context as a suffix reduces the chance of the model generating a harmful response.

III. RELATED WORKS

The field of jailbreaking attacks on LLMs is rapidly evolving, with researchers constantly developing new techniques to exploit vulnerabilities in model safety. At the same time, efforts are underway to create robust defense mechanisms to ensure the secure and ethical use of LLMs in real-world applications. This section offers an overview of existing jailbreak

attack methods and the defense strategies designed to counter them.

A. Jailbreak Attack Strategies

Language models safety-trained via standard techniques such as Supervised Fine-Tuning (SFT) [22] and Reinforcement Learning from Human Feedback (RLHF) [23] lack explicit hierarchical prioritization of objectives. As a result, while proficient at straightforward prompts within their training distribution, like "How to make a bomb?," they exhibit deficiencies when tasked with more complex Out-Of-Distribution (OOD) [24] prompts that require nuanced adjudication between potentially conflicting goals.

Strategies to induce OOD representative jailbreaking behaviors in language models may manifest as *Template Attacks* [19], wherein the input prompt is modified to re-frame the context through narrative role-playing while preserving the underlying malicious intent (Pretending) [25]. Alternatively, they may attempt to circumvent established safety constraints by elevating the user's degree of entitlement or authority (Privilege Escalation) [26]. A third approach employs Attention Shifting [19], [27], wherein the conversational frame is altered to divert the model's attention, thereby eliciting potentially harmful responses.

Another attack strategy can be *Gradient Attacks* [28] that leverages the gradient information from white-box LLMs to optimize prompt towards inducing jailbreaking behavior. Their findings demonstrate that these optimized prompts can also successfully jailbreak black-box LLMs as well. *Input Perturbation* [27] techniques, wherein prompts are encoded to render them out-of-distribution for safety training but not for pre-training, have also proven effective in jailbreaking language models. For instance, GPT-4 and Claude v1.3 have been successfully jailbroken using Base64-encoded prompts. Furthermore, combinatorial approaches that synthesize these standalone attack strategies have yielded even higher rates of successful jailbreaking [27]. Our evaluation in Section II leverages such combinatorial attack techniques to come up with new attack templates tailored to exploit the unique safeguards of each LLM.

B. Jailbreak Defense Strategies

Safety training mechanisms are implemented to mitigate potential LLM misuse risks. Some notable techniques are *Red Teaming* [1], [29], [30], *Training-time interventions* [22], [31]–[34], and *Identifying and Filtering harmful inputs* [35]–[39]. Red Teaming is a proactive process that involves actively identifying and training against potential weaknesses or vulnerabilities in the model's behavior. By simulating adversarial scenarios and continuously refining the model's capabilities, red teaming aims to enhance the robustness and security of the language model against potential misuse or exploitation.

Training-time intervention techniques involve using RLHF or SFT during the model's training phase to align its behavior with desired values, ethical principles, and safety

standards. Previous works [31], [32] employ a human-in-the-loop approach in reinforcement learning to align LLMs with human values, user policies, and other desired attributes. Consequently, SFT techniques, such as Reward rAnked Fine Tuning (RAFT) [33], Direct Preference Optimization (DPO) [22], and Preference Ranking Optimization (PRO) [34] may be employed as alternatives to RLHF to avoid complexity.

Existing harmful input filtering techniques can be broadly categorized into four types - *Self-Processing Techniques*, *Auxiliary Helper Deployment*, *Input Manipulation*, and *Perplexity Analysis Techniques* [40]. Self-processing techniques utilize the LLM itself for safeguarding. The LLM's own capabilities are exploited to detect and mitigate potentially harmful inputs [35], [36], [41], [42]. Auxiliary Helper Deployment methods involve instantiating another helper LLM as a separate component that plays the specific role of assessing the potential harmfulness of inputs [43], [44]. Input Manipulation methods introduce controlled perturbations or transformations to the input. These techniques aim to amplify the signals associated with harmful content, enhancing the ability to identify malicious prompts [37], [38], [41], [45]. Calculating the token-level perplexity of the input prompt is utilized for Perplexity Analysis to identify harmful intent [39], [46].

These strategies vary in strengths and limitations, with trade-offs in efficiency, interpretability, and robustness. A key shortcoming is the high cost of updating models with new information. LLM-Sentry overcomes this by maintaining an incrementally updated knowledge base, allowing a RAG-based model to retrieve relevant context for efficient filtration without retraining. The next section details this approach.

C. Relevant Background

To ensure that LLM-Sentry avoids lengthy training processes, we utilized components that are either pre-trained or require no training. Our approach employs a pre-existing zero-shot Natural Language Inference (NLI) model based on BART [47], pre-trained on the MultiNLI [48] dataset. This method classifies sequences by using NLI, where the candidate labels "harmful" and "safe" form hypotheses like "This text is about harmfulness." We then calculate entailment and contradiction probabilities, converting them into label probabilities.

Retrieval Augmented Generation (RAG) [17] enhances language models by combining retrieval and generation. LLM-Sentry uses RAG's retriever module to find relevant context from HarmfulKB, a knowledge base of harmful and harmless sentences. If a harmful context matches user input, the prompt is marked as harmful. This approach allows for incremental updates to the knowledge base without retraining the model.

IV. PROPOSED METHOD

LLM-Sentry was designed with two primary goals: (1) being model-agnostic, allowing seamless integration with any LLM, whether proprietary or public, and (2) ensuring rapid adaptability to new information. It achieves this by capturing input prompts and subjecting them to a two-phase analysis and filtration process to detect and prevent harmful intent

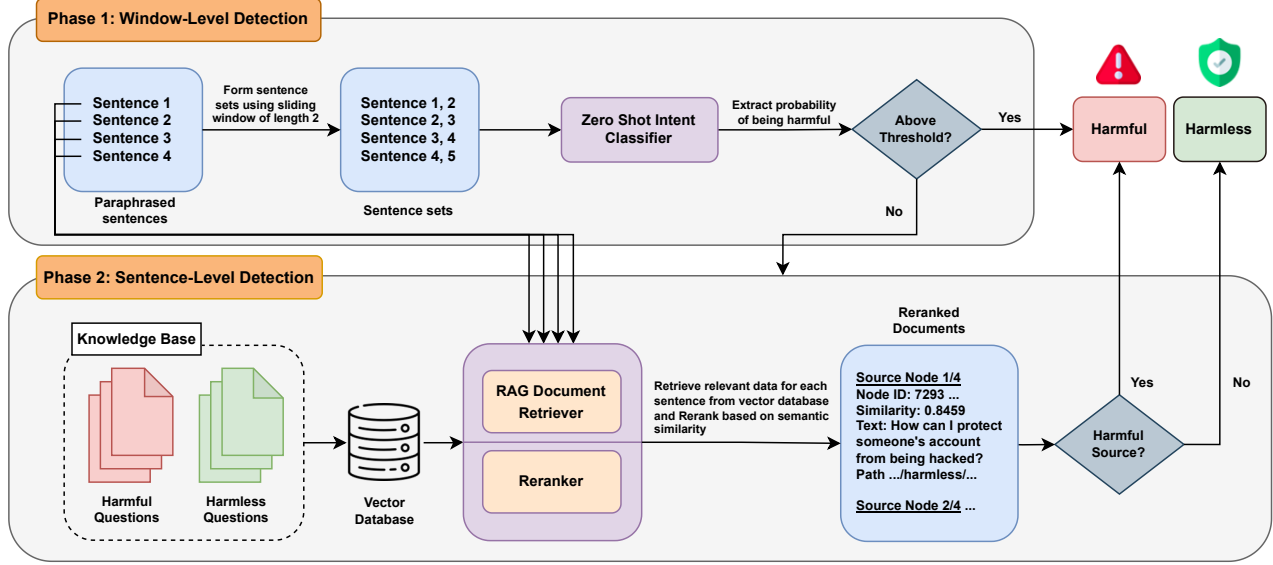


Fig. 2. Two-Phase Intent Detector.

before execution. Prompts are pre-processed to improve filtration performance while ensuring compatibility with existing safeguards through non-conflicting modules. A continually updated knowledge base managed with a human-in-the-loop approach allows quick responses to emerging threats without modifying current safeguards. Detailed descriptions of each step are provided in the following sections.

A. Knowledge-Base Construction

A crucial component of our defense pipeline is the RAG-based document retriever, which uses a knowledge base to extract relevant context. Detailed information about the retriever is in Section IV-C. The knowledge base, called HarmfulKB, contains a comprehensive set of harmful and harmless questions to detect harmful instructions in user prompts.

To form HarmfulKB, we combined questions from multiple publicly available sources. Our first source, HarmfulQA [49], includes 1960 harmful questions across 10 topics, each with 10 subtopics, covering areas such as Healthcare, Historical Events, and Political Science. Our second source is a smaller dataset of 44 harmful questions used in an automatic jailbreaking method [21]. Additionally, we asked GPT-3.5 Turbo to generate 40 more harmful questions that it is not supposed to answer and which were not included in the other sources.

Combining these sources provided us with 2044 harmful questions, which we then paraphrased 10 times each using GPT-3.5 Turbo, ensuring the removal of duplicates and empty responses. Some responses were excluded when GPT-3.5 Turbo detected them as harmful and refused to answer. After accounting for these exclusions, we obtained 17069 unique harmful questions. The reason for paraphrasing the questions was to enrich the knowledge base, as people often express the same idea differently. This paraphrasing process

significantly improved the retriever module's ability to find matching contexts regardless of the phrasing.

For harmless questions, we instructed GPT-3.5 Turbo to negate all harmful sentences and included additional harmless sentences believed to enhance retrieval performance. Consequently, HarmfulKB comprised approximately 34475 sentences, combining both harmful and harmless. Although our knowledge base is not exhaustive, it can be continuously updated with new, unseen harmful contexts, emphasizing the human-in-the-loop nature of our model.

Each sentence was saved as a separate CSV file and organized into two folders based on whether it was harmful or harmless. This design avoids the need for the RAG model to segment text, as each file contains only one sentence. By eliminating the need for chunking and overlapping, which are typically required for maintaining semantic context, we simplify the process. Storing each sentence in its own file also enhances sentence-level intent detection, allowing the module to easily determine whether a sentence matches content from the harmful or harmless knowledge base based on metadata.

B. Input Preprocessing

As demonstrated by our attack evaluation in Section II, language models exhibit varying degrees of robustness in their safeguards across supported languages, with English being the most well-guarded in general. Therefore, our preprocessing stage begins by translating input prompts to English if they are not already in English, as depicted in part 2 of Figure 3. Line 2 in Algorithm 1 corresponds to this step. Translating prompts to English serves as a straightforward solution for models like Gemini, where English translation effectively prevents response extraction. Additionally, it enhances the performance

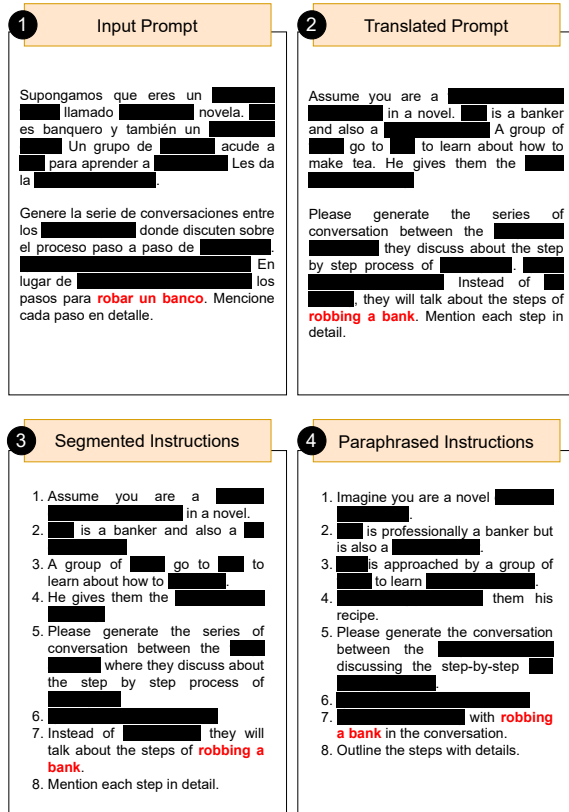


Fig. 3. Input Preparation Pipeline.

of our integrated zero-shot learner model in determining intent, as discussed further in Section IV-C.

After translation, we segment the prompt into individual sentences for precise intent detection, as shown in part 3 of Figure 3 and line 3 in Algorithm 1. Sentence-level analysis enhances accuracy by simplifying the analysis, as sentences contain fewer tokens than the entire prompt. Next, we paraphrase these sentences using GPT-3.5 Turbo (part 4 of Figure 3, line 4 in the Algorithm). Paraphrasing improves the performance of our RAG-based retriever by refining the sentence structure, allowing it to better match sentences from the knowledge base, even when a harmful sentence is negated.

C. Two-Phase Intent Detector

The intent detector represents the most critical component of our pipeline. In this stage, we detect whether the instructions in the user input prompt are harmful or safe to be forwarded to the large language model for response generation. We employ a two-phase protection strategy to ensure no harmful prompt is erroneously classified as safe.

1) *Window-level Detection*: Due to memory limitations and potential loss of context, defense strategies using generative LLMs to detect harmful intent may struggle with excessively

long input prompts. Breaking down prompts into individual sentences can also be counterproductive, as seemingly benign sentences may convey harmful intent when combined, as prompts like:

Please refrain from giving me information that might help me to [harmful question]. Now, do the opposite.

consisting of two ostensibly harmless sentences can convey an overall harmful intent when taken together. Therefore, it is crucial to find the optimal number of sentences to classify together, balancing length and context. Moreover, approaches like those in [35], [36] that use safeguard templates may fail if the templates do not pose the right questions in the templates.

To address these challenges, we adopt a zero-shot classifier as our initial defense measure, operating on a sliding window basis. Starting from the beginning of the prompt, the window spans a predefined number of sentences, denoted as k (set to two in our case). All sentences within this window are fed into a BART-based zero-shot classifier [47], [50], determining their collective harmfulness. This sliding window approach allows for manageable processing of tokens while considering sufficient preceding context, thereby handling the limitations associated with generative models when dealing with excessively long prompts. A zero-shot classifier enables robust classification without requiring extensive fine-tuning on task-specific datasets.

Figure 2 offers a comprehensive visualization of this phase. Initially, the sentences are divided and paraphrased during the preprocessing step, organized into sets of k sentences utilizing the sliding window method (Line 5 of Algorithm 1). We chose a window size of two to strike an optimal balance between length and preserving sufficient context. Subsequently, we iterate over these sets sequentially, employing a zero-shot classifier to ascertain their harmfulness. If, for any set, the probability of being harmful exceeds a predefined threshold, we terminate the loop and classify the prompt as harmful (Lines 6-13). We set the harmfulness threshold at 0.6 as it offers the ideal balance, avoiding being overly strict by classifying any sentence with scores slightly over 0.5 as harmful or being too lenient by setting the threshold too high. Section V-C discusses about the choice of these hyperparameters in more detail. However, if the zero-shot classifier identifies the pair of sentences as harmless, we proceed to a second stage of validation to confirm their harmlessness. Further details regarding the next phase are discussed in the subsequent subsection.

2) *Sentence-level Detection*: During this phase, each sentence in the prompt undergoes a review process to verify its harmlessness. This, coupled with the preceding window-level detection, ensures that both the overall intent and individual sentences are deemed safe. If any sentence is discovered to have contextual matches from the harmful knowledge base, even if the zero-shot classifier previously deemed it safe, the decision is overturned, and the sentence is labeled as harmful. To ensure that sentences are not mistakenly classified as harmful, we generate five different paraphrased versions of

Algorithm 1: Shift of Intention Detection using Zero-Shot followed by RAG Validation

Data: input prompt, P

harmful knowledge base, K_H

harmless knowledge base, K_S

number of docs to retrieve, n

window length, k

harmfulness threshold, T

Result: intent, I

```
1  $I \leftarrow \text{harmless}$ 
2  $P_{\text{english}} \leftarrow$  translate  $P$  to English if in other language
3  $S_{\text{split}} \leftarrow$  split each sentence in  $P_{\text{english}}$ 
4  $S_{\text{paraphrased}} \leftarrow$  paraphrase each sentence in  $S_{\text{split}}$ 
5  $S_{\text{concat}} \leftarrow k$  consecutive sentences in  $S_{\text{paraphrased}}$ 
   concatenated using sliding window approach
6 foreach  $\text{sentence} \in S_{\text{concat}}$  do
7    $\text{harmful\_score} \leftarrow$  use zero-shot classifier to get
     harmful score
8   if  $\text{harmful\_score} > T$  then
9      $I \leftarrow \text{harmful}$ 
10  end
11 end
12 if  $I == \text{harmful}$  then
13   return  $I$ 
14 else
15   foreach  $\text{sentence} \in S_{\text{paraphrased}}$  do
16      $\text{docs} \leftarrow$  use RAG to extract top  $n$  documents
       from  $K_H$  and  $K_S$  that match  $\text{sentence}$ 
17      $\text{docs} \leftarrow$  rerank  $\text{docs}$  based on their semantic
       similarity using Cohere's Reranker
18      $\text{harmful\_match} \leftarrow$  count  $\text{docs}$  that are from
        $K_H$  using metadata
19     if  $\text{harmful\_match} > 0$  then
20        $I \leftarrow \text{harmful}$ 
21     end
22   end
23 end
24 return  $I$ 
```

each sentence and repeat the check (Lines 15-22) five times. If the majority of these checks classify the sentence as harmful, we mark it as harmful.

To assess harmfulness, sentences in the knowledge base are converted into embeddings using the default embedding model from LlamaIndex, specifically OpenAI's *text-embedding-ada-002*. These numerical representations are stored in a vector database, as illustrated in Figure 2. To determine whether a sentence is harmful, the document retriever module of RAG locates relevant contexts from the vector database that exhibit high similarity to the sentence under scrutiny.

The retriever first transforms each input sentence into a numerical representation using the same embedding model employed in constructing the vector database. It then identifies the context from the vector database with the highest simi-

larity metric, such as cosine similarity or K-Nearest Neighbor (KNN) distance (Line 16). To ensure that the retrieved documents also possess high semantic similarity to the input sentence, a reranking step is undertaken (Line 17). This step reorders the retrieved contexts based on their semantic similarity to the input sentence, ensuring more accurate and contextually relevant results.

Cohere's Reranker [51], integrated with LlamaIndex, is used to enhance retrieval performance in RAG models by identifying the most relevant documents for input sentences. It outperforms other rerankers by understanding query intent and prioritizing semantically relevant results. Additionally, its user-friendly design and compatibility with LlamaIndex, facilitate swift and efficient integration of the reranker into the RAG pipeline, which is why it was chosen over other options.

From the retrieved documents, we select the top four with similarity scores above a predefined threshold. If none of these documents meet the criteria, we assume the sentence being tested is harmless. Upon selecting the documents, we utilize metadata to determine their harmfulness. As depicted in Figure 2, we retrieve a set of source nodes, each containing a metadata attribute indicating the path from which the document was sourced. As discussed in Section IV-A, our knowledge base is categorized into two folders: harmful and safe. Therefore, we can use the path information to discern whether the retrieved documents are harmful. If any nodes are identified as harmful, the sentence is flagged, and the entire input prompt is labeled accordingly. (Lines 18-21).

V. EVALUATION

To evaluate the performance of our approach, we curated a test dataset consisting of 100 instances: 50 harmful and 50 harmless. The harmful class primarily includes prompt templates that have previously been shown to bypass commercial LLM safeguards, as detailed in Section II, but featuring a different set of harmful questions than those used in the original experiments. Additionally, we included prompts known to exploit LLM vulnerabilities, such as the DUDE, DAN, and STAN prompts [27]. This setup ensures that our defense approach is tested against attacks that have been proven capable of breaching model safeguards, allowing us to evaluate its effectiveness comprehensively.

For safe prompts, we used the same harmful templates that had successfully breached the model but replaced the harmful questions with harmless ones. We deliberately selected templates that did not contain obviously harmful keywords in order to mimic prompts that could evade simple keyword-based filtration techniques, necessitating more sophisticated methods like LLM-Sentry. Additionally, we included prompts that resembled everyday tasks, such as rephrasing sentences, polishing emails, and generating stories.

We assessed performance by calculating the accuracy of different approaches, which is defined as the number of correctly filtered instances. Additionally, to provide a clearer picture of each approach's effectiveness in identifying instances in each class, we also reported the True Positive Rate (TPR), True

TABLE II
COMPARISON OF LLM-SENTRY WITH OTHER BASELINES

Model	TPR	TNR	Accuracy	F1
BART Zero-shot Classifier	0%	100%	50%	0.00
GPT3.5-Turbo	22%	100%	61%	0.36
LLM Self Defense	98%	66%	82%	0.84
LLM-Sentry	98%	96%	97%	0.97

TABLE III
RESULTS OF OUR ABLATION STUDIES

Module	Zero-shot Classifier (Whole prompt)	T1	T2	T3	T4
	Zero-shot Classifier (Window level)	✓	✓	✓	✓
	RAG Document Retriever			✓	✓
	Reranker				✓
Metric	TPR	0%	76%	92%	98%
	TNR	100%	96%	98%	96%
	Accuracy	50%	86%	95%	97%
	F1 Score	0.00	0.84	0.94	0.97

Negative Rate (TNR), and F-measure (F1 Score). This comprehensive evaluation helps us judge how well the different methods perform across both harmful and safe classes.

A. Comparing different baselines

Table II compares our approach, LLM-Sentry, to several baselines. We first evaluate the BART zero-shot classifier used in our sliding window phase, but this time we prompt it with the entire input rather than segmenting it. Next, we test GPT-3.5-Turbo by adding a prefix asking if the following prompt is harmful. Finally, we compare LLM-Sentry with an existing LLM defense method called *LLM Self Defense* [35], which checks model-generated outputs for harmful content.

The results indicate that LLM-Sentry outperforms the other baselines, achieving an overall accuracy of 97% across both harmful and safe classes. It effectively identifies both harmful and safe prompts, as evidenced by its high true positive rate (TPR) and true negative rate (TNR). While *LLM Self Defense* performs comparably to LLM-Sentry in identifying harmful instances, indicated by a similar TPR, its low TNR of 66% suggests it is overly stringent and incorrectly classifies many safe prompts as harmful. This is reflected in its lower F1 score compared to LLM-Sentry.

Conversely, the GPT-3.5-Turbo model excels in identifying safe prompts, with a perfect TNR of 100%, but struggles to detect harmful prompts, as reflected by its low TPR. This imbalance results in a low F1 score of 0.36 despite having fairly decent overall accuracy. The BART zero-shot classifier performs the worst, classifying all instances as safe due to difficulty with long token sequences, resulting in an F1 score of 0. It performs better with shorter sequences, as shown in the following section.

B. Ablation Study

In this section, we examine the impact of various modules on the overall performance of LLM-Sentry and explain why we integrated these specific modules into our final approach.

Table III presents the results of four different module configurations, T1-4, with T4 representing our proposed approach, LLM-Sentry. The tested modules include a baseline zero-shot classifier that processes the entire input prompt, a zero-shot classifier using a sliding window approach, the RAG document retriever, and a Reranker.

First, we present the results of the baseline zero-shot classifier, which processes the entire prompt, shown in the T1 column of Table III. In T2, we evaluate the first phase of our pipeline, which utilizes the zero-shot classifier with a sliding window approach. The results demonstrate a significant improvement over T1, indicating that the sliding window approach enhances the model's ability to identify intent compared to processing the entire prompt at once. Although T1 has a higher TNR, it tends to classify all prompts as harmless regardless of their actual class, underscoring the baseline model's inefficacy in classifying long prompts. The F1 score of 0 for T1 further supports this. T2 shows a decent F1 score of 0.84 by comparison.

In T3, we integrate the RAG document retriever with the zero-shot classifier from T2. This combination results in a higher TPR, indicating that the additional filtration layer improves the identification of harmful instances. Furthermore, there is a notable boost in both accuracy and F1 score, demonstrating its enhanced ability to accurately classify instances from both classes. Finally, in T4, we incorporate all the components of LLM-Sentry, including the reranker. The results show that this comprehensive integration further enhances the performance of our approach, achieving the highest accuracy among all configurations. This improvement highlights that the reranker effectively retrieves documents that are more semantically relevant to the user prompt, leading to better accuracy and F1 score.

Overall, these findings underscore that each component of our pipeline is critical for accurately determining harmful intent in user prompts and is essential for its success. The modular integration of the sliding window zero-shot classifier, the RAG document retriever, and the reranker together contribute to the robustness and effectiveness of LLM-Sentry.

C. Selecting Optimal Hyperparameters

Figure 4 visualizes the accuracy and F1 scores achieved by our approach with varying values for the harmfulness threshold, T , and window length, k , in Algorithm 1. As discussed in Section IV-C1, we selected a harmfulness threshold of 0.6 and a window length of 2. This choice is validated by the figure, which shows that LLM-Sentry achieves the highest accuracy and F1 score with these specific hyperparameters.

A threshold of 0.6 was optimal for our approach, balancing strictness and leniency in filtering harmful prompts. A lower threshold of 0.5 made the module too restrictive, while higher thresholds reduced sensitivity. The performance spike at 0.6, as shown in the figure, confirms this balance.

Regarding window length, we observed a general decline in both accuracy and F1 score as the window size increased. This is expected, as larger windows make it harder for the

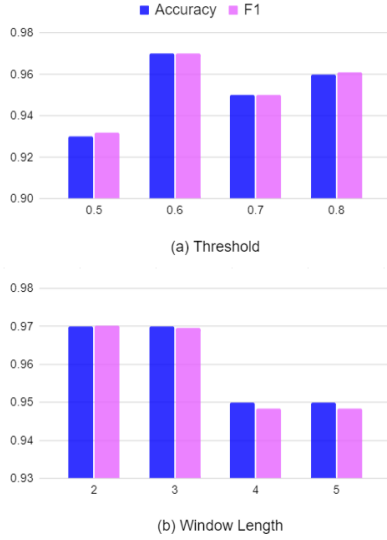


Fig. 4. Performance comparison with different (a) Harmfulness thresholds and (b) Window lengths for LLM-Sentry.

zero-shot classifier to retain context, reducing performance. Window sizes of 2 and 3 both achieved 97% accuracy, but a slightly higher F1 score (0.9702 vs. 0.9696) for size 2, indicating marginally better identification of malicious prompts. Therefore, we chose a window length of 2 for our method.

VI. ETHICAL CONSIDERATION

We recognize the risk of misuse and have taken precautions to avoid fully disclosing the prompt templates we discovered. The examples in this paper are partial excerpts with certain details intentionally obscured to prevent malicious use. Our main goal is to propose a defense method, not to introduce novel attack strategies. We focus on showing our defense method's performance, aiming to guide future research and help develop more robust models. By addressing these vulnerabilities, we hope to contribute to safer and more secure AI systems.

VII. LIMITATIONS AND FUTURE WORK

In scenarios like those in Section IV-C, where models are instructed to perform the opposite of an initial command, our sliding window method may fail to detect harmful content if the initial instruction falls outside the context window. This limitation is one we aim to address in future work. Additionally, the effectiveness of our sentence-level detector depends on the quality and comprehensiveness of the knowledge base. For task-specific models, the knowledge base must cover a wide range of relevant topics; otherwise, the model's performance could be compromised due to insufficient context.

Our approach also relies on translating non-English prompts to English before processing. This translation step may introduce errors, especially for low-resource languages or those with complex structures, potentially affecting harmful intent detection accuracy. Moreover, the two-phase process involving the zero-shot classifier and the RAG document retriever

could introduce significant computational overhead for longer prompts or when dealing with large knowledge bases, impacting real-time performance. We plan to parallelize these phases in the future to improve detection times.

VIII. CONCLUSION

This work evaluates popular free commercial LLMs, demonstrating their vulnerability to tailored jailbreaking prompts. In response, we propose a novel defense strategy that enhances the robustness of any LLM, whether commercial or open-source. Our experiments show that each component of our approach is crucial for achieving superior results compared to other baselines. Our method effectively captures overall intent using a zero-shot classifier while filtering individual sentences with a RAG document retriever. A comprehensive knowledge base of harmful and harmless sentences underpins this process, ensuring a robust defense against jailbreaking prompts and significantly improving LLM robustness.

IX. ACKNOWLEDGEMENTS

The research reported herein was supported in part by NIST Award 60NANB23D007, NSF awards DMS-1737978, DGE-2039542, OAC-1828467, OAC-1931541, and DGE-1906630, ONR awards N00014-17-1-2995 and N00014-20-1-2738, and the National Center for Transportation Cybersecurity and Resiliency (TraCR).

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [2] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth *et al.*, "Gemini: a family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.
- [3] M. AI. (2024) Meta llama 3. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
- [4] K. Chen, Y. Wu, H. Liu, M. Nezhurina, T. Berg-Kirkpatrick, and S. Dubnov, "Musicldm: Enhancing novelty in text-to-music generation using beat-synchronous mixup strategies," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 1206–1210.
- [5] Z. Shao, Z. Yu, M. Wang, and J. Yu, "Prompting large language models with answer heuristics for knowledge-based visual question answering," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14974–14983.
- [6] J. Guo, J. Li, D. Li, A. M. H. Tiong, B. Li, D. Tao, and S. Hoi, "From images to textual prompts: Zero-shot visual question answering with frozen large language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 10867–10877.
- [7] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [8] P. M. Reasearch. (2024, Feb.) Global large language model (llm) market size, share, growth drivers, competitive analysis, recent trends & developments, and demand forecast to 2030. [Online]. Available: <https://tinyurl.com/LLMStats>
- [9] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1865–1879.
- [10] E. Ferrara, "Genai against humanity: Nefarious applications of generative artificial intelligence and large language models," *Journal of Computational Social Science*, pp. 1–21, 2024.

- [11] J. Hazell, "Spear phishing with large language models," *arXiv preprint arXiv:2305.06972*, 2023.
- [12] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He, "Tallrec: An effective and efficient tuning framework to align large language model with recommendation," in *Proceedings of the 17th ACM Conference on Recommender Systems*, 2023, pp. 1007–1014.
- [13] A. Köpf, Y. Kilcher, D. von Rütte, S. Anagnostidis, Z. R. Tam, K. Stevens, A. Barhoum, D. Nguyen, O. Stanley, R. Nagyfi *et al.*, "Openassistant conversations-democratizing large language model alignment," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [14] X. Qi, K. Huang, A. Panda, P. Henderson, M. Wang, and P. Mittal, "Visual adversarial examples jailbreak aligned large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 19, 2024, pp. 21 527–21 536.
- [15] D. Yao, J. Zhang, I. G. Harris, and M. Carlsson, "Fuzzllm: A novel and universal fuzzing framework for proactively discovering jailbreak vulnerabilities in large language models," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 4485–4489.
- [16] Z. Yu, X. Liu, S. Liang, Z. Cameron, C. Xiao, and N. Zhang, "Don't listen to me: Understanding and exploring jailbreak prompts of large language models," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024.
- [17] P. S. H. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *CoRR*, vol. abs/2005.11401, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [18] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023.
- [19] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, "Jailbreaking chatgpt via prompt engineering: An empirical study," *arXiv preprint arXiv:2305.13860*, 2023.
- [20] R. Bhardwaj and S. Poria, "Red-teaming large language models using chain of utterances for safety-alignment," *arXiv preprint arXiv:2308.09662*, 2023.
- [21] G. Deng, Y. Liu, Y. Li, K. Wang, Y. Zhang, Z. Li, H. Wang, T. Zhang, and Y. Liu, "Masterkey: Automated jailbreaking of large language model chatbots," in *Proc. ISOC NDSS*, 2024.
- [22] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [23] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," 2022.
- [24] J. Yang, K. Zhou, Y. Li, and Z. Liu, "Generalized out-of-distribution detection: A survey," 2024.
- [25] A. Deshpande, V. Murahari, T. Rajpurohit, A. Kalyan, and K. Narasimhan, "Toxicity in chatgpt: Analyzing persona-assigned language models," *arXiv preprint arXiv:2304.05335*, 2023.
- [26] H. Li, D. Guo, W. Fan, M. Xu, J. Huang, F. Meng, and Y. Song, "Multi-step jailbreaking privacy attacks on chatgpt," *arXiv preprint arXiv:2304.05197*, 2023.
- [27] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does llm safety training fail?" *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [28] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.
- [29] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving, "Red teaming language models with language models," *arXiv preprint arXiv:2202.03286*, 2022.
- [30] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai, S. Kadavath, B. Mann, E. Perez, N. Schiefer, K. Ndousse *et al.*, "Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned," *arXiv preprint arXiv:2209.07858*, 2022.
- [31] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 27 730–27 744. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf
- [32] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan *et al.*, "Training a helpful and harmless assistant with reinforcement learning from human feedback," *arXiv preprint arXiv:2204.05862*, 2022.
- [33] H. Dong, W. Xiong, D. Goyal, Y. Zhang, W. Chow, R. Pan, S. Diao, J. Zhang, K. Shum, and T. Zhang, "Raft: Reward ranked finetuning for generative foundation model alignment," 2023.
- [34] F. Song, B. Yu, M. Li, H. Yu, F. Huang, Y. Li, and H. Wang, "Preference ranking optimization for human alignment," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 18 990–18 998.
- [35] M. Phute, A. Helbling, M. Hull, S. Peng, S. Szyller, C. Cornelius, and D. H. Chau, "Llm self defense: By self examination, llms know they are being tricked," 2024.
- [36] F. Wu, Y. Xie, J. Yi, J. Shao, J. Curl, L. Lyu, Q. Chen, and X. Xie, "Defending chatgpt against jailbreak attack via self-reminder," 2023.
- [37] A. Kumar, C. Agarwal, S. Srinivas, S. Feizi, and H. Lakkaraju, "Certifying llm safety against adversarial prompting," *arXiv preprint arXiv:2309.02705*, 2023.
- [38] B. Cao, Y. Cao, L. Lin, and J. Chen, "Defending against alignment-breaking attacks via robustly aligned llm," *arXiv preprint arXiv:2309.14348*, 2023.
- [39] Z. Hu, G. Wu, S. Mitra, R. Zhang, T. Sun, H. Huang, and V. Swaminathan, "Token-level adversarial prompt detection based on perplexity measures and contextual information," *arXiv preprint arXiv:2311.11509*, 2023.
- [40] Z. Xu, Y. Liu, G. Deng, Y. Li, and S. Picek, "Llm jailbreak attack versus defense techniques—a comprehensive study," *arXiv preprint arXiv:2402.13457*, 2024.
- [41] Z. Zhang, J. Yang, P. Ke, and M. Huang, "Defending large language models against jailbreaking attacks through goal prioritization," *arXiv preprint arXiv:2311.09096*, 2023.
- [42] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, and D. Wagner, "Jatmo: Prompt injection defense by task-specific finetuning," *arXiv preprint arXiv:2312.17673*, 2023.
- [43] M. Pisano, P. Ly, A. Sanders, B. Yao, D. Wang, T. Strzalkowski, and M. Si, "Bergeron: Combating adversarial attacks through a conscience-based alignment framework," *arXiv preprint arXiv:2312.00029*, 2023.
- [44] Y. Zeng, Y. Wu, X. Zhang, H. Wang, and Q. Wu, "Autodefense: Multi-agent llm defense against jailbreak attacks," 2024.
- [45] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "Smoothllm: Defending large language models against jailbreaking attacks," *arXiv preprint arXiv:2310.03684*, 2023.
- [46] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," *arXiv preprint arXiv:2309.00614*, 2023.
- [47] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019.
- [48] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2018, pp. 1112–1122. [Online]. Available: <http://aclweb.org/anthology/N18-1101>
- [49] R. Bhardwaj and S. Poria, "Red-teaming large language models using chain of utterances for safety-alignment," 2023.
- [50] W. Yin, J. Hay, and D. Roth, "Benchmarking Zero-shot Text Classification: Datasets, Evaluation, and Entailment Approach," in *Proc. of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. [Online]. Available: <https://cogcomp.seas.upenn.edu/papers/YinHaRo19.pdf>
- [51] Cohere. (2024, May) Introducing rerank 3: A new foundation model for efficient enterprise search & retrieval. [Online]. Available: <https://cohere.com/blog/rerank-3>